BriskSeed: Online History-Guided Reuse for Accelerating Dynamic Approximate Nearest Neighbor Search

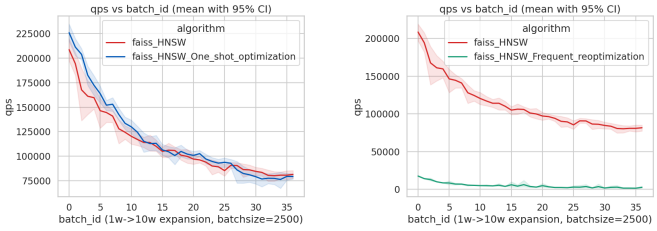
Hongru Gao, Shuhao Zhang*, Xiaofei Liao, Hai Jin
National Engineering Research Center for Big Data Technology and System,
Services Computing Technology and System Lab, Cluster and Grid Computing Lab,
School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China
{hrgao, shuhao_zhang, xfliao, hjin}@hust.edu.cn

***Abstract*—Approximate Nearest Neighbor Search (ANNS) is a core primitive in modern data systems. Graph-based indices can deliver high search efficiency on static datasets, but sustaining such performance becomes challenging when the indexed data continuously evolve. As insertions and deletions reshape the local graph structure, auxiliary information used at search time, such as quantization encodings, pruning statistics, and routing hints, can quickly lose utility. Reusing them blindly may hurt recall or waste search effort, while rebuilding them online introduces non-trivial overhead. In this paper, we present *BriskSeed*, a pluggable optimization layer for dynamic ANNS that reuses past successful search outcomes as lightweight search states, called *seeds*, to improve search performance under continuous updates. Instead of discarding stale auxiliary information or refreshing it after every update, *BriskSeed* judiciously stores, retrieves, and validates seeds through a unified online workflow. Specifically, it maintains a compact two-tier seed storage guided by utility, retrieves seeds via exact-key and signature-based access paths, and validates each reuse decision with a lightweight acceptance-and-fallback policy. This design allows *BriskSeed* to accelerate the search without modifying the underlying ANNS index or assuming a specific graph backend. We evaluate *BriskSeed* using CANDOR-Bench, a streaming ANNS benchmark framework, across multiple dynamic ANNS backends through a unified plugin interface. Experiments show that *BriskSeed* consistently improves end-to-end search efficiency under high update rates while preserving competitive recall, demonstrating that stale-aware search-state reuse is an effective optimization primitive for dynamic ANNS.**

***Index Terms*—Approximate Nearest Neighbor Search, Data Structure, Graph Index.**

I. Introduction

Approximate Nearest Neighbor Search (ANNS) is a core primitive in modern data systems, including Retrieval-Augmented Generation (RAG), recommendation, and anomaly detection [1]. Among existing ANNS index families, graph-based indices such as HNSW [2], [3] are widely adopted because they offer strong recall-latency trade-offs at scale. Their search process relies on iterative graph traversal from vertices to their neighbors, where random memory access and cache misses often dominate query latency in practice. To mitigate these challenges, prior work has explored a broad range of optimizations: some efforts make distance computation and traversal faster through optimized storage layout, quantization techniques, etc [1], [4], [5], while others reduce the amount of computation through pruning and distance estimation [6]–[8].



(a) One-shot optimization (b) Frequent re-optimization

Fig. 1: Trade-offs in dynamic ANNS: one-shot optimization provides only short-term gains, while aggressive frequent re-optimization hurts overall query throughput.

Despite substantial progress, most existing optimizations are developed for static snapshots. In practical streaming systems, vectors are continuously inserted and deleted, causing the index topology and data distribution to evolve over time. This leads to a temporal mismatch: pre-computed search hints, i.e., auxiliary information or structures used to accelerate queries, may be effective at time t but become stale at time $t + \Delta$, reducing alignment with the current index and degrading query efficiency. The effect is especially severe in high-rate ingestion workloads, where updates arrive continuously and drift accumulates quickly. A representative case is real-time payment fraud detection [9], [10], where incoming transactions are embedded and matched against a continuously updated behavioral database via ANNS under millisecond-level decision SLAs. In such systems, the index must absorb new transactions while serving latency-critical queries. Significant performance degradation across updates is unacceptable because delayed retrieval directly increases fraud leakage risk and financial loss.

To alleviate update-induced performance decay, dynamic ANNS faces a fundamental efficiency-maintenance dilemma, as illustrated in Figure 1. If search hints are computed once

and left unmaintained during updates, they may provide short-term gains in early rounds, but their benefit decays over time, causing query throughput to progressively fall back toward baseline performance (Figure 1a). In contrast, if the system frequently rebuilds or globally re-optimizes them, the maintenance overhead is prohibitive and can dominate end-to-end runtime, limiting overall throughput gains (Figure 1b). Consequently, the key challenge is not merely how to accelerate search, but how to keep acceleration *continuously valid* under ongoing updates without incurring prohibitive maintenance cost.

In this paper, we present *BriskSeed*, an ANNS plugin for continuously accelerating query performance under dynamic updates. The key observation behind *BriskSeed* is that historical search outcomes do not lose their utility uniformly after updates. In many streaming workloads, batch queries still exhibit substantial top-k overlap across updates, query accesses are often highly skewed, and different queries within the same batch can exhibit very different degrees of result stability. By exploiting these characteristics, *BriskSeed* avoids repeatedly executing queries from scratch by selectively reusing past successful search outcomes as lightweight search states, called *seeds*. Subsequently, seeds can guide candidate generation across update rounds and be managed online efficiently without resorting to one-shot offline optimization or frequent global re-optimization. This approach reduces redundant graph traversal and distance computation, yielding higher end-to-end query efficiency under dynamic updates.

A key design in *BriskSeed* is to organize seed reuse as an online store–retrieve–validate workflow. First, *BriskSeed* maintains a compact two-tier seed storage, where each *seed* records historical results together with lightweight freshness and utility signals. The high-utility layer keeps a small set of valuable seeds for fast access, while the semantic-shared layer provides bounded coverage for long-tail historical results. Second, at query time, *BriskSeed* retrieves seeds through exact-key lookup and signature-based scan over high-utility seeds or signature-based probing over semantic-based seeds, and uses the retrieved seeds to guide bounded neighbor expansion and candidate generation. Third, before returning the seed-guided top-k results, *BriskSeed* applies a lightweight validation gate; if reuse is unreliable, the query falls back to the original backend search procedure. After each processing round, effective seeds are selectively written back, and low-value seeds are evicted, allowing the seed storage to track workload skew and index drift with bounded maintenance overhead.

Different from existing static ANNS optimizations that accelerate queries on a fixed index snapshot, the emphasis of *BriskSeed* is to sustain query-throughput gains after the index has been changed by continuous updates. The design of *BriskSeed* is also complementary to existing dynamic ANNS works that focus on update throughput or graph maintenance. *BriskSeed* operates as a pluggable post-update query acceleration layer, decoupled from backend-specific graph internals and deployable across different ANNS backends. We evaluate *BriskSeed* on CANDOR-Bench [11], a streaming ANNS benchmark, across three state-of-the-art ANNS backends. The experimental results confirm the effectiveness of *BriskSeed* under high update rates.

TABLE I: Summary of terminologies

Symbol	Description
$\mathcal{D}$	Vector dataset.
$\mathcal{D}_r$	Vector dataset state after applying the batch updates at round r.
$\widehat{N}_k(q)$	Approximate top-k result set of query q.
$N_k^*(q)$	Exact top-k nearest-neighbor set of query q.
$\mathrm{Recall@}k(q)$	Recall of the approximate top-k result for query q.
$\mathcal{U}_r$	Batch updates at round r.
$\mathcal{Q}_r$	Batch queries executed at round r.
R	The number of rounds we evaluate.
$T_{\mathrm{search}}(q)$	Search latency of query q.
$T_{\mathrm{maintain}}(\mathcal{U}_r)$	Maintenance time cost triggered by batch updates $\mathcal{U}_r$.
$\mathrm{QPS}(R)$	Average query throughput within R rounds.
τ	Tolerance threshold for Recall@k.

This paper makes the following contributions:

- We identify stale auxiliary guidance as a major cause of search-efficiency decay in dynamic ANNS, and formulate the problem as an amortized trade-off between search acceleration and guidance maintenance.
- We present *BriskSeed*, an ANNS plugin to improve query performance under dynamic updates by using seeds, while remaining decoupled from backend-specific index internals.
- We evaluate *BriskSeed* on CANDOR-Bench through extensive experiments. The results show that *BriskSeed* enhances end-to-end query efficiency while maintaining competitive recall under high update rates.

The rest of this paper is organized as follows. Section 2 formalizes the problem and preliminaries. Section 3 presents the *BriskSeed* design. Section 4 provides an analytical method for *BriskSeed* instantiation. Section 5 describes implementation details. Section 6 reports evaluation results. Section 7 discusses related work, and Section 8 concludes this work.

II. Problem Formulation and Motivation

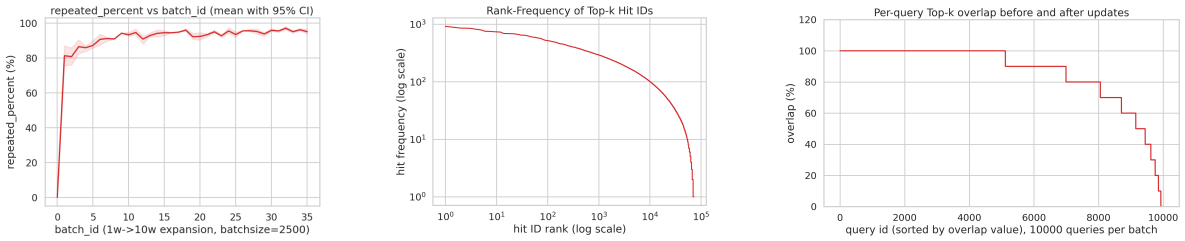
This section first formulates the problem in Section 2.1 and then presents key observations in Section 2.2. Finally, Section 2.3 discusses the main challenges in designing *BriskSeed*. The related terminologies are listed in Table I.

A. Problem Statement

We begin with the standard Approximate Nearest Neighbor Search (ANNS) definition and then extend it to the dynamic setting considered in this paper.

Definition 1 (ANNS). Given a vector dataset $\mathcal{D} \subset \mathbb{R}^d$, a query $q \in \mathbb{R}^d$, and an integer k, ANNS returns an approximate top-k set $\widehat{N}_k(q)$ that approximates the exact nearest-neighbor set $N_k^*(q)$ under a given distance metric. The search quality metric is $\mathrm{Recall@}k$:

$$\mathrm{Recall@}k(q) = \frac{|\widehat{N}_k(q) \cap N_k^*(q)|}{k}. \tag{1}$$



(a) Top-k overlap across rounds (b) Skewness of returned vector IDs (c) Per-query top-k overlap

Fig. 2: Key observations in dynamic ANNS workloads.

Definition 1 captures the static setting, where the dataset and index remain fixed during queries. In practical systems, however, vectors are continuously inserted and deleted, causing the ANNS index to evolve over time. To model this setting, we adopt the *serialized round-based execution* model used in CANDOR-Bench [11].

Definition 2 (Dynamic ANNS). Dynamic ANNS operates as a sequence of update-query rounds. At each round r, the system first applies a batch of updates $\mathcal{U}_r$ (insertions/deletions) to transform the previous dataset $\mathcal{D}_{r-1}$ into $\mathcal{D}_r$, and then executes a batch of queries $\mathcal{Q}_r$ on $\mathcal{D}_r$. The index structure is updated online across rounds.

In this setting, many optimizations developed for static ANNS become less effective as updates accumulate. As their effectiveness decays, query throughput drops and recall may also deteriorate. Frequent re-optimization is not a viable solution either, because its maintenance overhead can dominate end-to-end cost, as shown in Figure 1b. This dilemma motivates the following problem.

Problem Statement. Our goal is to maximize end-to-end query throughput under continuous updates, while preserving search quality comparable to the baselines. Since query acceleration is useful only when its maintenance overhead is affordable, we account for both query execution time and update-triggered maintenance time. Over R update-query rounds, we measure end-to-end query throughput as:

$$\mathrm{QPS}(R) = \frac{\sum_{r=1}^{R} |\mathcal{Q}_r|}{\sum_{r=1}^{R} \sum_{q \in \mathcal{Q}_r} T_{\mathrm{search}}(q) + \sum_{r=1}^{R} T_{\mathrm{maintain}}(\mathcal{U}_r)}. \tag{2}$$

where $T_{\mathrm{search}}(q)$ denotes the execution time of query q, and $T_{\mathrm{maintain}}(\mathcal{U}_r)$ denotes the maintenance time triggered by $\mathcal{U}_r$.

Accordingly, the optimization objective is to maximize $\mathrm{QPS}(R)$ subject to an average recall constraint:

$$\max \ \mathrm{QPS}(R) \quad \text{s.t.} \quad \overline{\mathrm{Recall@}k} \geq \tau \cdot \overline{\mathrm{Recall@}k}^{\mathrm{baseline}}. \tag{3}$$

where $\overline{\mathrm{Recall@}k}$ is the average recall over all queries in the R rounds, and $\tau \in (0, 1]$ specifies the required recall ratio relative to the baseline.

B. Key Observations

To better understand the behavior of dynamic ANNS workloads, we conduct a preliminary study and summarize three observations.

Observation 1: Recent search outcomes often remain informative across consecutive rounds. For the same query batch, the top-k results often remain highly overlapping across consecutive update rounds. As shown in Figure 2a, the overlap ratio of top-k results quickly exceeds 80% after the first few batches and remains around 95% thereafter. This result suggests that although accumulated updates may gradually reduce the effectiveness of acceleration techniques, they do not immediately cause large changes in the query results. Therefore, recent high-quality search results can still provide useful guidance for later queries, even though they should not be blindly returned as final answers.

Observation 2: Returned vector frequencies are highly skewed. Across batch queries, the vector IDs appearing in top-k results are not uniformly distributed. As shown in Figure 2b, a small fraction of vectors appears frequently in query results, while most vectors are rarely returned. This skew suggests that historical search results are not equally useful: prioritizing results related to frequently returned vectors is likely to provide more benefit than treating all past outcomes uniformly.

Observation 3: Result stability varies across queries. Within a query batch, different queries can respond very differently to updates. As shown in Figure 2c, we observe that roughly 50% of queries keep their top-k results unchanged before and after batch updates, while nearly 10% of queries experience at least 50% changes in their top-k results. This variation suggests that historical results cannot be assumed to be reliable for all queries, even when the overall batch-level overlap remains high.

C. Design Challenges

Although the observations above reveal clear opportunities for dynamic ANNS acceleration, exploiting them efficiently and robustly under continuous updates remains non-trivial, with the following challenges:

Challenge 1: How to store and represent historical search results? Observation 1 shows that recent top-k results can remain stable across nearby update rounds. However, storing complete query results for all past queries is neither memory-efficient nor necessary. The system must decide what information from previous queries should be stored, how to represent it in a compact form, and how to attach simple signals that reflect its freshness and usefulness. It must also support bounded

retention and eviction, so that outdated or low-value records do not accumulate as updates continue.

Challenge 2: How to guide new queries with stored results? Stored historical results are useful only if they can be matched to later queries and used to reduce search cost. This is straightforward when a query repeats a previous one, but harder when the query is only similar to past queries. The system must therefore decide which stored results are relevant to the current query and how they can guide the search process. Observation 2 further shows that returned vector frequencies are highly skewed, suggesting that stored results should not be treated uniformly during retrieval and reuse.

Challenge 3: How to validate the new results derived from stored results? Even if historical search results can be retrieved for a new query, the results derived from them may not be reliable after updates. Observation 3 shows that result stability varies significantly across queries, so blindly trusting stored results may lead to low-quality generated results. The system must decide whether to accept the generated top-k results or fall back to the original search path. This validation step must be lightweight; otherwise, its cost can offset the benefit of reusing stored results.

Motivated by these challenges, we next present *BriskSeed* and elaborate on how it addresses Challenges 1–3.

III. BriskSeed Design

This section presents the full *BriskSeed* design. Section 3.1 introduces the design overview. Section 3.2 describes the overall pipeline of *BriskSeed*. Section 3.3 gives a complexity analysis of *BriskSeed*.

A. Design Overview

Motivated by the observations and challenges in Section 2, we design ***BriskSeed*** as a three-layer online framework for accelerating dynamic ANNS. *BriskSeed* is built on a simple principle: historical search outcomes, referred to as *seeds*, should be reused only when they can be represented compactly, activated selectively, and trusted safely under continuous updates. To this end, *BriskSeed* combines three layers that map directly to the three challenges in Section 2: Seed Storage Layer, Seed-Guided Search Layer, and Reliability Control Layer, as shown in Figure 3.

(1) Seed Storage Layer (addresses Challenge 1). This layer determines how historical outcomes are represented, retained, and refreshed under bounded memory. To exploit the high top-k overlap across nearby rounds, *BriskSeed* organizes reusable history in a two-tier seed storage: a high-utility tier for highly valuable outcomes and a semantic-shared tier for compact long-tail coverage. Each seed stores historical top-k IDs together with lightweight metadata needed for retrieval and maintenance. In this way, past outcomes are encoded as reusable guidance rather than final answers, while retention and eviction remain explicitly budgeted.

(2) Seed-Guided Search Layer (addresses Challenge 2). This layer determines how the stored results guide the search. For each query, *BriskSeed* first retrieves seeds through exact lookup, signature-based scan, or bounded semantic probing, and then converts the selected seed into provisional candidates through bounded neighbor expansion. The guidance strategy is tier-aware: high-utility seeds use one-hop expansion by default, while semantic-shared seeds use guarded multi-hop expansion only when one-hop evidence is insufficient. This design enables efficient reuse under skewed workloads while preventing uncontrolled search overhead.

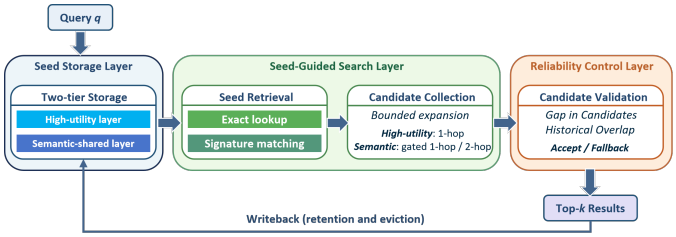


Fig. 3: BriskSeed design overview.

(3) Reliability Control Layer (addresses Challenge 3). This layer ensures that reuse improves end-to-end efficiency without degrading recall. Provisional candidates are checked by a lightweight acceptance policy, where acceptance requires both strong local candidate quality and consistency with the seed's historical neighborhood. If either signal is weak, *BriskSeed* falls back to the baseline search path. This explicit validation gate ensures that acceleration remains beneficial while bounding recall risk.

These three layers are co-designed rather than independent. The seed storage layer determines what is reusable, the seed-guided search layer determines how reuse enters search, and the reliability control layer determines when provisional candidates are trustworthy enough to be accepted. Together, they form a bounded online reuse framework that aligns directly with the three design challenges and remains robust under continuous updates.

B. Unified Online Reuse Pipeline

In this subsection, we detail the modules of BriskSeed and their interactions in the unified online reuse pipeline. Important symbols are summarized in Table II.

1) Seed Representation and Storage:

a) Seed representation: We first define the representation of a seed, and then describe how such seeds are organized in the storage.

Each seed records a past successful search outcome together with lightweight metadata:

$$r = \langle \mathbf{z}, \kappa, \sigma, n_b, \rho \rangle. \tag{4}$$

where $\mathbf{z} \in \mathbb{N}^k$ denotes the historical top-k result IDs, which serve as reuse anchors for future queries. κ is the source query key, used for exact lookup when the same query appears again. σ is a compact source-query signature, used to retrieve seeds generated by similar but non-identical queries. n_b records the seed birth time (when the seed was created), and ρ records the successful reuse count.

The metadata serve different purposes. The query key κ supports low-cost exact retrieval for recurring queries. The signature σ supports similarity-based retrieval for new or related queries. The birth time n_b and reuse count ρ summarize freshness and past effectiveness, and are used for seed ranking, retention, and eviction.

b) Query signature: To support low-overhead similarity-based retrieval, *BriskSeed* computes a compact locality-preserving signature for each query. Given a query vector q, *BriskSeed* uses fixed random projections to generate a 64-bit signature:

$$\sigma(q)_i = \mathbf{1}[\mathbf{a}_i^\top q \geq 0], \quad i = 1, \ldots, 64, \tag{5}$$

where $\mathbf{a}_1, \ldots, \mathbf{a}_{64}$ are fixed random projection vectors. For a seed generated by a historical query q_r, we store $\sigma = \sigma(q_r)$ as its source-query signature.

This signature is used only by *BriskSeed* for seed retrieval and does not participate in the backend ANN search. Compared with full-vector comparison, signature comparison requires only bit operations. In particular, the similarity between a query q and a seed r can be estimated by Hamming similarity:

$$\mathrm{sim}_\sigma(q, r) = 1 - \frac{\mathrm{Ham}(\sigma(q), \sigma_r)}{64}, \tag{6}$$

where σ_r is the signature stored in seed r. This allows *BriskSeed* to find seeds generated by similar queries without storing or comparing full query vectors.

c) Two-tier seed storage: Given the seed representation above, *BriskSeed* organizes seeds in a two-tier storage consisting of a *high-utility layer* and a *semantic-shared layer*. The high-utility layer is a bounded storage with capacity H. It is reserved for seeds with high expected reuse value, such as those that are recent, frequently reused, or repeatedly beneficial. Each seed in this layer is indexed by its source query key κ and also keeps its source-query signature σ. This allows the layer to support recurring queries through exact-key access, while also enabling queries without an exact key match to later compare against high-utility seeds using their signatures. The capacity H is intentionally kept small, so that the subsequent retrieval stage can scan this layer with low overhead without maintaining an additional similarity index.

The semantic-shared layer provides broader coverage for long-tail historical results under a fixed storage budget. It is organized as S buckets, each storing at most m seeds. Seeds are assigned to buckets according to compact keys derived from their source-query signatures. Unlike the high-utility layer, the semantic-shared layer keeps a wider set of lower-frequency seeds and bounds memory growth through per-bucket capacity limits.

A single-layer design is practically insufficient. A per-query storage that keeps one seed for each historical query can accurately serve repeated queries, but its memory footprint grows with the number of past queries, and it offers limited benefit when the current query has no exact key match. A fully shared storage limits memory usage but mixes frequently useful seeds with many rarely useful ones, making high-value seeds harder to retain and retrieve efficiently. *BriskSeed* therefore separates these two roles: the high-utility layer keeps a small, capacity-bounded set of valuable seeds for fast retrieval, while the semantic-shared layer provides compact coverage for long-tail historical results.

TABLE II: Symbols used in BriskSeed design.

Symbol	Description
Storage and Retrieval	
q	Incoming query.
$r = \langle \mathbf{z}, \kappa, \sigma, n_b, \rho \rangle$	Record of a seed: historical top-k IDs, source query key, signature, birth time, source query ID, and accepted-reuse count.
$\sigma(q)$	Signature of query q.
$\mathrm{sim}_\sigma(q, r)$	Hamming similarity between q and a seed r.
$S,\ m,\ P,\ H$	Semantic bucket count, per-bucket seed limit, probing width, and high-utility layer capacity.
$u(r)$	Score of seed r for seed ranking.
Validation	
$\hat{\mathbf{z}}$	Provisional top-k set.
$\mathrm{gap}_k(q)$	Boundary gap between the k-th and $(k+1)$-th candidates
$\mathrm{overlap}(q)$	Overlap between $\mathbf{z}$ and $\hat{\mathbf{z}}$
Writeback and Maintenance	
$\mathrm{keep}(r)$	Keep score of seed r for eviction.

2) Seed-guided Search:

a) Seed retrieval: At query time, *BriskSeed* first retrieves a seed that is likely to be useful for the current query. Given a query q, *BriskSeed* computes its query key $\kappa(q)$ and its 64-bit signature $\sigma(q)$. Retrieval proceeds in three steps: exact lookup in the high-utility layer, signature-based scan in the high-utility layer, and bounded probing in the semantic-shared layer.

Exact lookup in the high-utility layer. *BriskSeed* first checks whether $\kappa(q)$ exists in the high-utility layer. If a matching seed is found, *BriskSeed* selects it as the retrieved seed. This path is effective for recurring queries, because the seed was generated by the same query key and is therefore expected to be well aligned with the current query.

Signature-based scan in the high-utility layer. If exact lookup misses, *BriskSeed* scans the high-utility layer using the query signature. For each seed r in this layer, *BriskSeed* compares $\sigma(q)$ with the seed's source-query signature σ_r using Hamming similarity $\mathrm{sim}_\sigma(q, r)$ listed in Equation 6. Since the high-utility layer has a small fixed capacity H, this scan only requires $O(H)$ 64-bit comparisons. Each candidate seed is then ranked by a lightweight score that combines signature similarity and past reuse effectiveness:

$$u(r) = \lambda \cdot \mathrm{sim}_\sigma(q, r) + (1 - \lambda) \cdot \widetilde{\rho}_r, \tag{7}$$

where $\widetilde{\rho}_r$ is the normalized successful reuse count of seed r. The highest-ranked seed is selected only if its score exceeds a retrieval threshold θ_{ret}. Otherwise, *BriskSeed* proceeds to the semantic-shared layer. This step allows queries without an exact key match to still benefit from high-utility seeds generated by similar historical queries.

Bounded probing in the semantic-shared layer. If the high-utility layer cannot provide a confident seed, *BriskSeed*

probes the semantic-shared layer. Let $s(q)$ denote the bucket key derived from $\sigma(q)$, such as a short prefix or band of the signature. *BriskSeed* first probes the primary bucket $s(q)$ and then probes at most $P-1$ nearby buckets whose keys have small Hamming distance from $s(q)$:

$$\mathcal{N}(q) = \{s(q)\} \cup \mathrm{Near}_{P-1}(s(q)),\ P \ll S. \tag{8}$$

Because each bucket stores at most m seeds, this step inspects at most Pm seeds. All inspected seeds are ranked using the same scoring mechanism listed in Equation 7, and the highest-ranked seed is selected.

This seed retrieval process is intentionally ordered from high-confidence and low-cost paths to broader but less certain paths. Exact lookup serves repeated queries, signature-based scan exploits high-utility seeds for related queries, and semantic-shared probing allows long-tail historical results to be reused under bounded retrieval cost.

b) Candidate collection: Given a selected seed, *BriskSeed* builds a bounded candidate set from the backend graph index using the seed's historical result IDs as starting points. Specifically, the nodes recorded in the seed are first included in the candidate set. *BriskSeed* then expands from these nodes by following graph edges, so that the candidate set contains both the historical top-k results and nearby nodes that may become relevant after recent updates.

The expansion depth depends on where the selected seed comes from. If the seed is retrieved from the high-utility layer, *BriskSeed* performs one-hop expansion by default. This applies to both exact-key matches and signature-based matches in the high-utility layer. Because high-utility seeds are selected from a small set of recent and repeatedly useful seeds, they usually have higher retrieval confidence. Therefore, one-hop expansion is usually sufficient to collect strong candidates while keeping the search cost low.

For a seed retrieved from the semantic-shared layer, *BriskSeed* uses a slightly more conservative expansion strategy. Since semantic-shared seeds are kept for broader long-tail reuse opportunities, the selected seed may be less tightly aligned with the current query. *BriskSeed* therefore first performs one-hop expansion and evaluates the best candidate obtained. If this candidate is not sufficiently close to the query, *BriskSeed* expands one more hop; otherwise, it stops after one hop. The expansion depth is capped at two hops to prevent rapid frontier growth and keep the candidate collection bounded.

After candidate collection, *BriskSeed* ranks all collected candidates and truncates them to form a provisional top-k result, denoted by $\hat{\mathbf{z}}$. This provisional result is then passed to the validation stage described next.

3) Candidate Validation: Since the provisional top-k result $\hat{\mathbf{z}}$ is derived from local expansion around historical results, *BriskSeed* does not return it blindly. Instead, it validates $\hat{\mathbf{z}}$ using two inexpensive signals: the top-k boundary gap within the collected candidate set and the overlap with the selected seed.

a) Top-k boundary gap: After candidate collection, *BriskSeed* has already ranked all collected candidates. Let c_k denote the k-th candidate in this ranking and c_{k+1} denote the next closest candidate outside $\hat{\mathbf{z}}$. *BriskSeed* computes the normalized boundary gap as

$$\mathrm{gap}_k(q) = \frac{d(q, c_{k+1}) - d(q, c_k)}{d(q, c_k) + \epsilon}, \tag{9}$$

where $\epsilon > 0$ is a small constant for numerical stability. This signal measures whether the provisional top-k result has a clear boundary within the collected candidate set. A small $\mathrm{gap}_k(q)$ means that the boundary between included and excluded candidates is ambiguous, making the provisional result sensitive to small candidate changes. In such cases, *BriskSeed* treats the provisional result conservatively.

b) Overlap with the selected seed: The boundary gap only reflects the ranking structure within the collected candidate set. It does not indicate whether the selected seed still provides relevant guidance for the current query. Therefore, *BriskSeed* also compares the provisional top-k result $\hat{\mathbf{z}}$ with the historical top-k result $\mathbf{z}$ stored in the selected seed:

$$\mathrm{overlap}(q) = \frac{|\hat{\mathbf{z}} \cap \mathbf{z}|}{k}. \tag{10}$$

This signal reflects the relevance between the provisional result and the historical result region represented by the selected seed. A low overlap does not necessarily mean that $\hat{\mathbf{z}}$ is incorrect, but it indicates that the current result has deviated substantially from the selected seed. As a result, the confidence of this reuse decision becomes lower.

c) Acceptance rule: BriskSeed accepts the provisional result only when the top-k boundary is sufficiently clear and the overlap with the selected seed is sufficiently high. Otherwise, *BriskSeed* falls back to the original backend search procedure. Intuitively, a small $\mathrm{gap}_k(q)$ suggests that the provisional top-k boundary is ambiguous, while a low $\mathrm{overlap}(q)$ suggests that the selected seed may provide weak guidance for the current query. Together, these two signals allow *BriskSeed* to use seed-guided results conservatively, improving efficiency while protecting recall with little validation overhead.

4) Writeback Mechanism and Seed Update: After each query finishes, *BriskSeed* writes the final top-k results back to update seeds. The final result may come from an accepted seed-guided path or from the original backend search after fallback. In both cases, it provides fresh search information that can be used to update existing seeds or create new ones. The writeback step updates seed statistics, refreshes stored results when appropriate, and maintains the two-tier seed store under bounded capacity.

If the query already has a seed in the high-utility layer, *BriskSeed* updates the corresponding record under the same query key. The stored top-k result is refreshed, the source-query signature is updated if needed, and the reuse statistics are adjusted according to whether the seed-guided result was accepted. If the query does not have a high-utility entry, *BriskSeed* creates a new seed from the final top-k result and

inserts it into the semantic-shared layer according to the bucket key derived from its source-query signature.

When the target semantic-shared bucket is full, *BriskSeed* evicts the seed with the lowest keep score:

$$\mathrm{keep}(r) = \xi\,\rho - (1-\xi)\,\mathrm{age}_r, \quad \mathrm{age}_r = n - n_b. \tag{11}$$

where ρ is the successful reuse count and age_r is the age of seed r. This score favors seeds that have been reused successfully and penalizes old seeds that have not shown recent value. By evicting low-score seeds within each bucket, *BriskSeed* bounds memory usage while keeping seeds that are more likely to be useful again.

Writeback also allows seeds to move between the two layers over time. A seed in the semantic-shared layer can be promoted to the high-utility layer when it is repeatedly reused successfully or receives a high keep score. Conversely, a seed in the high-utility layer can be demoted to the semantic-shared layer when it becomes old or shows low reuse effectiveness. Since the high-utility layer has a fixed capacity H, promotion may trigger eviction or demotion of the lowest-value high-utility seed. Overall, this update policy keeps the high-utility layer focused on a small set of valuable seeds, while the semantic-shared layer maintains broader long-tail coverage under bounded memory.

C. Theoretical Analysis

This subsection analyzes why *BriskSeed* can improve query throughput and when such improvement is feasible. Let T_b denote the average per-query latency of the baseline under a fixed recall target.

1) Time Complexity Comparison: For each query, *BriskSeed* first retrieves a seed from the two-tier seed storage. In the high-utility layer, exact-key lookup takes $O(1)$ time and signature-based scan consumes $O(H)$ lightweight 64-bit Hamming comparisons. When it proceeds to the semantic-shared layer, the semantic probing inspects at most P buckets and m seeds per bucket, yielding $O(Pm)$ cost. Therefore, seed retrieval is bounded by $O(H + Pm)$.

Given a retrieved seed, the candidate collection performs one-hop or guarded two-hop neighbor expansion under degree bound M, incurring $O(kM)$ or $O(kM^2)$ time cost. Validation is bounded by the candidate set size, while writeback takes at most $O(m + H)$ time for bucket maintenance and possible high-utility layer adjustment. Hence, the worst-case reuse time cost is

$$T_{\text{reuse}} = O(H + Pm + kM^2). \tag{12}$$

All terms are controlled by system parameters rather than the dataset size.

Let p_{acc} be the probability that the seed-guided result is accepted. The expected per-query latency of *BriskSeed* is

$$T_{\text{BriskSeed}} = T_{\text{reuse}} + (1 - p_{\text{acc}})T_b. \tag{13}$$

Thus, the query latency reduction over the baseline is

$$\Delta = T_b - T_{\text{BriskSeed}} = p_{\text{acc}}T_b - T_{\text{reuse}}. \tag{14}$$

Here, T_b depends on the index size N; for HNSW, it is commonly modeled as $T_b = O(\log N)$. In contrast, T_{reuse} is bounded by fixed system parameters H, P, m, k, and M, which are much smaller than N. Therefore, *BriskSeed* can improve query throughput over the baseline when seed-guided results are accepted frequently enough, especially on large indices.

IV. Instantiation of BriskSeed

Implementing *BriskSeed* requires instantiating the parameters that govern the efficiency-recall trade-off under dynamic updates. Since the expansion depth is fixed by design (no more than two hops), we focus on three storage parameters:

$$\mathbf{x} = (H, S, m), \tag{15}$$

where H is the capacity of the high-utility layer, and S and m are the number of semantic-shared buckets and the per-bucket seed capacity, respectively.

Our objective is to maximize end-to-end query throughput under a recall constraint, which is equivalent to minimizing the expected per-query time cost:

$$\max_{\mathbf{x}} \frac{\mathrm{QPS}_{\mathrm{BriskSeed}}(\mathbf{x})}{\mathrm{QPS}_{\mathrm{baseline}}} = \frac{T_b}{\mathbb{E}[T](\mathbf{x})} \quad \Longleftrightarrow \quad \min_{\mathbf{x}} \mathbb{E}[T](\mathbf{x}), \tag{16}$$

where T_b is the average query latency of the baseline.

A. Performance Model

For each query, *BriskSeed* first attempts high-utility reuse. If this attempt is not accepted, it attempts semantic-shared reuse; if that also fails, the query falls back to the baseline search. Let $p_h(H)$ denote the probability that the high-utility reuse succeeds, and let $p_s(m)$ denote the probability that the semantic-shared reuse is accepted. Let T_h and T_s denote the time costs of attempting high-utility and semantic-shared reuses, respectively. The expected per-query latency is

$$\mathbb{E}[T] = T_h + \big(1 - p_h(H)\big)\Big(T_s + \big(1 - p_s(m)\big)T_b\Big). \tag{17}$$

Both reuse attempts consist of seed retrieval, bounded expansion, and validation. Since validation only compares the provisional top-k result with the seed and checks the top-k boundary, its cost is marginal compared with seed inspection and local expansion; we absorb it into the constant terms.

For high-utility reuse, exact-key lookup incurs $O(1)$ cost, while signature-based matching may scan up to H seeds. The subsequent one-hop expansion visits at most kM neighbors, where M is the graph degree bound. Let T_{io}, T_{score}, and T_{comp} denote the calibrated cost of one memory access, one seed scoring operation, and one distance computation, respectively. We use a conservative upper-bound approximation

$$T_h \approx H(T_{\mathrm{io}} + T_{\mathrm{score}}) + kM(T_{\mathrm{io}} + T_{\mathrm{comp}}). \tag{18}$$

For semantic-shared reuse, with a fixed probing width P_0, at most $P_0 m$ seeds are inspected. Since semantic-shared seeds may trigger guarded two-hop expansion, we approximate

$$T_s \approx P_0 m(T_{\mathrm{io}} + T_{\mathrm{score}}) + kM^2(T_{\mathrm{io}} + T_{\mathrm{comp}}). \tag{19}$$

Following the empirical observation that semantic-shared acceptance increases and then saturates as more seeds are inspected, we model

$$p_s(m) = 1 - e^{-\gamma m}, \qquad \gamma > 0. \tag{20}$$

Substituting Equations (18) and (19) into Equation (17) yields

$$\mathbb{E}[T] \approx aH + \big(1 - p_h(H)\big)\big(bm + ce^{-\gamma m} + d\big), \tag{21}$$

where $a, b, c, d > 0$ are calibrated constants, and terms independent of H and m are absorbed into the constant part.

B. Analytical Instantiation

We model the skewed workload using a Zipf distribution over Q query groups, where each group represents recurring or similar queries that can potentially share seeds. Let f_r denote the fraction of queries belonging to the r-th most frequent group:

$$f_r = \frac{r^{-\zeta}}{\sum_{u=1}^{Q} u^{-\zeta}}, \qquad \zeta > 0. \tag{22}$$

where ζ controls the skewness of the workload.

The cumulative mass of the top-H ranks provides a simple approximation of the high-utility acceptance probability:

$$p_h(H) = \sum_{r=1}^{H} f_r = \frac{\sum_{r=1}^{H} r^{-\zeta}}{\sum_{u=1}^{Q} u^{-\zeta}}. \tag{23}$$

Using the standard integral approximation, we further obtain

$$p_h(H) \approx \frac{H^{1-\zeta} - 1}{Q^{1-\zeta} - 1}. \tag{24}$$

This approximation captures the main effect of increasing H: a larger high-utility layer covers more head queries, but also increases scan cost.

Let B be the total seed-storage budget. With S fixed, the budget constraint is

$$H + Sm = B, \qquad H, m \in \mathbb{Z}_+. \tag{25}$$

Hence the instantiation problem reduces to

$$\min_{H,m} \ \mathbb{E}[T](H, m) \quad \text{s.t.} \quad H + Sm = B. \tag{26}$$

Since the feasible integer set induced by $H + Sm = B$ is finite, the exact optimum can be obtained by enumeration. For deployment, however, we derive a continuous initializer from a local first-order Taylor expansion approximation, and then evaluate only a small integer neighborhood around it. Around an operating point H_0, we linearize the high-utility acceptance probability as

$$\begin{aligned} p_h(H) &\approx \beta_0 + \beta_1 H, \\ \beta_0 = p_h(H_0) - H_0 p_h'(H_0), &\ \beta_1 = p_h'(H_0) > 0. \end{aligned} \tag{27}$$

Similarly, around m_0, we linearize

$$e^{-\gamma m} \approx e^{-\gamma m_0} - \gamma e^{-\gamma m_0}(m - m_0). \tag{28}$$

These approximations are used in a neighborhood of (H_0, m_0) to obtain a tractable local model. Substituting $H = B - Sm$ from Equation (25) into Equation (21) yields a quadratic surrogate in m:

$$\widetilde{\mathbb{E}}[T](m) = C_2 m^2 + C_1 m + C_0, \tag{29}$$

where

$$\begin{aligned} C_2 &= S\beta_1\big(b - c\gamma e^{-\gamma m_0}\big), \\ C_1 &= -aS + \big(1 - \beta_0 - \beta_1 B\big)\big(b - c\gamma e^{-\gamma m_0}\big) \\ &\quad + \beta_1 S\Big(d + c\big(1 + \gamma m_0\big)e^{-\gamma m_0}\Big). \end{aligned} \tag{30}$$

If $b > c\gamma e^{-\gamma m_0}$, the surrogate is strictly convex and its continuous minimizer is

$$m^\star_{\text{cont}} = -\frac{C_1}{2C_2}, \qquad H^\star_{\text{cont}} = B - Sm^\star_{\text{cont}}. \tag{31}$$

In practice, we project $(H^\star_{\text{cont}}, m^\star_{\text{cont}})$ to feasible integers and evaluate a small neighborhood to obtain final configurations.

V. Implementation

In this section, we describe the implementation of *BriskSeed* and discuss its remaining limitations.

A. Plugin Architecture

We implement *BriskSeed* as a backend-agnostic plugin layer rather than tightly integrating it into a specific ANNS backend. The plugin exposes a unified interface for index setup, insertion, deletion, and query execution, while each backend is connected through a lightweight adapter that maps backend-specific operations to this interface. This design keeps integration minimally invasive: *BriskSeed* does not modify the index construction, update policy, or native search procedure of the backend, and all seed management logic is implemented outside the backend core. The same benchmark pipeline is used to evaluate both the original backend and its *BriskSeed*-enabled variant under identical workload and system settings. All deployment parameters are externalized in configuration files, e.g., the high-utility layer capacity, the number of semantic-shared buckets, and the per-bucket seed capacity. An explicit switch, `BriskSeed_mode`, enables or disables *BriskSeed* on the same backend, and the benchmark runner logs the effective configuration with the dataset, batch settings, and backend identifier at startup. This separation between system logic and experiment configuration enables deterministic reruns and fair comparisons across different ANNS backends.

B. Limitations

The current implementation still leaves some limitations. First, *BriskSeed* works best in common dynamic ANNS scenarios where the indexed database is large and each update batch modifies only a small fraction of it, such as online recommendation. When updates are large relative to the existing data, seed effectiveness may be significantly weakened, causing more fallbacks and limited speedups. Second, the current prototype maintains seeds in memory and updates them online, but does not persist seed states across process restarts. After a restart, previously accumulated seed statistics

TABLE III: Datasets used in evaluation.

Dataset	Type	Dimension	DataSize	QuerySize
SIFT	Image	128	1,000,000	10,000
OpenImages	Image	512	1,000,000	10,000
Msong	Audio	420	992,272	200
Glove	Text	100	1,192,514	200

are not retained. Finally, *BriskSeed* currently targets single-node deployments; distributed seed placement and consistency across index partitions are left for future work.

VI. Evaluation

This section first presents the experimental setups and then assesses *BriskSeed* from the following aspects:

- Does *BriskSeed* improve query throughput while maintaining recall when plugging into existing ANNS systems? (Section 6.2)
- How much additional memory does *BriskSeed* introduce? (Section 6.3)
- How important is the validation gate for balancing throughput improvement and recall preservation? (Section 6.4)
- How do the key storage and retrieval parameters affect performance? (Section 6.5)
- How does *BriskSeed* behave under different numbers of threads? (Section 6.6)

A. Experimental Setup

Test-bed. Experiments are conducted on a Linux Server (Ubuntu 22.04) with two Intel Xeon Gold 5117 CPUs (each with 14 cores, 2.00GHz) and 512GB memory.

Baselines. We evaluate *BriskSeed* on CANDOR-Bench [11] by deploying it on three representative ANNS systems and comparing their performance with and without *BriskSeed* under the same dynamic benchmark protocol. (1) HNSW [3], one of the most widely-used ANNS systems; (2) SymphonyQG [4], a recent static ANNS system that combines graph index with RabitQ quantization technique; and (3) Wolverine [12], a dynamic ANNS system designed for online updates. We denote the corresponding *BriskSeed*-enabled variants as HNSW-*BriskSeed*, SymphonyQG-*BriskSeed*, and Wolverine-*BriskSeed*.

Datasets. We choose four million-scale datasets, `SIFT`, `OpenImages`, `Msong`, and `Glove`. These datasets cover image, audio, and text embeddings with different dimensionalities. Table III summarizes their statistics.

Metrics. We use end-to-end query throughput and Recall@k as the primary metrics. Throughput is measured in queries per second (QPS), computed as the number of executed queries divided by the total post-update query processing time. Unless otherwise stated, we report Recall@10 against the exact nearest-neighbor ground truth.

Parameters. For graph-based backends, the graph out-degree is set to $M = 16$; the construction beam width is set to *efConstruction*$= 120$; and the default search beam width is set to *efSearch*$= 80$. These settings provide a balanced recall–latency tradeoff on million-scale datasets.

Evaluation Protocol. For each dataset, we use half of the vectors to build the initial index and treat the remaining vectors as dynamic updates, which are applied in batches of 10,000 vectors. After each update batch, we execute the corresponding query batch and record QPS and Recall@10, following the round-based update-then-query execution model of CANDOR-Bench. Each experiment is repeated three times, and we report the average result.

B. QPS Performance and Recall

C. Memory Usage

D. Ablation Study

E. Parameter Study

F. Scaling-up Study

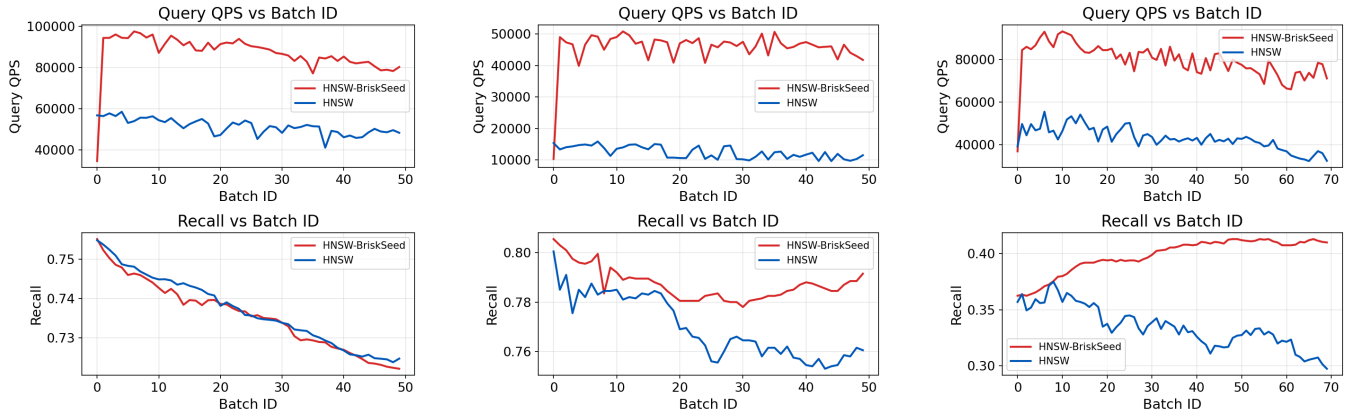
VII. Related Work

A. Acceleration Technique for Graph-based ANNS

Existing work can be divided into two categories. The first category preserves the original graph search and improves efficiency through system-level optimization. GraphReorder [13] improves cache locality by reordering vertices based on their access frequency, while MARGO [14] optimizes graph layout by weighting edges according to their importance along monotonic search paths. However, such methods usually require expensive reconstruction and are therefore difficult to apply repeatedly in dynamic scenarios. SymphonyQG [4], FLASH [5], and VSAG [1] improve efficiency through redundant neighborhood storage and quantization-based encoding. Nevertheless, their reliance on offline-learned codebooks or centroids makes them costly to maintain under continuous updates and prone to recall degradation. The second category accelerates search by explicitly reducing distance computation. ADSampling [7], DDC [15], and DADE [16] employ partial-dimensional distances to determine whether full distance computation can be skipped. SkipComputing [8] further improves pruning accuracy through lower-bound verification, while Finger [6] applies the residual-based approximation. Although these methods directly target the dominant cost of high-dimensional search, they introduce branch-heavy execution or additional preprocessing dependencies, limiting robustness in dynamic settings.

B. Dynamic Graph-based ANNS

Recent studies on dynamic graph-based ANNS can be broadly divided into two categories. The first category focuses on efficiently maintaining index freshness under continuous updates. FreshDiskANN [17] extends DiskANN [2] to dynamic settings by repairing connectivity loss in a fully connected manner. By relaxing edge trimming, it preserves newly added edges and maintains index accuracy after updates. LM-DiskANN [18] further reduces the memory footprint of disk-native dynamic graph indexing by storing complete routing information in each node. Wolverine [12] repairs



(a) SIFT (b) Msong (c) Glove

Fig. 4: QPS and Recall comparison between the baseline and *BriskSeed* on three datasets.

monotonic search paths disrupted by deletions to maintain search quality during updates. These systems primarily aim to support efficient updates while maintaining connectivity and recall in dynamic scenarios. The second category focuses on reducing graph construction cost. Yang et al. [19] revisit the construction pipelines of RNG and NSWG and accelerate graph construction through a refinement-before-search strategy. Relative NN-Descent [20] combines NN-Descent with RNG-style edge selection to directly construct a search graph more efficiently. MIRAGE-ANNS [21] combines incremental and refinement-based strategies to balance construction speed and search quality. While these methods effectively reduce the cost of building graph indices, they do not target query acceleration after updates. In contrast, our work focuses on improving post-update query efficiency in dynamic settings by exploiting historical search information.

VIII. Conclusion

In this work, we study query acceleration for ANNS under dynamic updates. Existing acceleration methods face a fundamental tension: search hints constructed for static snapshots quickly become stale as updates arrive, while frequent global re-optimization incurs high maintenance overhead and can offset the intended throughput gains. To address this challenge, we propose *BriskSeed*, a lightweight dynamic acceleration plugin that turns static hinting into an online reuse–validate–refresh loop. *BriskSeed* maintains bounded reusable seed states and combines exact reuse on hot paths with semantic retrieval, bounded neighbor expansion, and fallback validation to preserve recall while improving end-to-end efficiency. We further present an analytical model for practical parameter instantiation under budget constraints. Extensive experiments show that *BriskSeed* consistently improves throughput, preserves competitive recall, and delivers robust performance under dynamic update workloads.

References

[1] X. Zhong, H. Li, J. Jin, M. Yang, D. Chu, X. Wang, Z. Shen, W. Jia, G. Gu, Y. Xie, X. Lin, H. T. Shen, J. Song, and P. Cheng, "Vsag: An optimized search framework for graph-based approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5017–5030, 2025.

[2] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[3] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[4] Y. Gou, J. Gao, Y. Xu, and C. Long, "Symphonyqg: Towards symphonious integration of quantization and graph for approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 3, no. 1, Feb. 2025. [Online]. Available: https://doi.org/10.1145/3709730

[5] M. Wang, H. Wu, X. Ke, Y. Gao, Y. Zhu, and W. Zhou, "Accelerating graph indexing for anns on modern cpus," *Proc. ACM Manag. Data*, vol. 3, no. 3, Jun. 2025. [Online]. Available: https://doi.org/10.1145/3725260

[6] P. Chen, W.-C. Chang, J.-Y. Jiang, H.-F. Yu, I. Dhillon, and C.-J. Hsieh, "Finger: Fast inference for graph-based approximate nearest neighbor search," in *Proceedings of the ACM Web Conference 2023*, ser. WWW '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 3225–3235. [Online]. Available: https://doi.org/10.1145/3543507.3583318

[7] J. Gao and C. Long, "High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations," *Proc. ACM Manag. Data*, vol. 1, no. 2, Jun. 2023. [Online]. Available: https://doi.org/10.1145/3589282

[8] Z. Song, B. Wang, and X. Yang, "Accelerating high-dimensional ann search via skipping redundant distance computations," *Proc. ACM Manag. Data*, vol. 3, no. 6, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3769753

[9] Microsoft, "Real-time credit card fraud detection with vector search," Azure Cosmos DB Samples, 2025, accessed: 2025-04-13. [Online]. Available: https://learn.microsoft.com/en-us/samples/azurecosmosdb/cosmos-fabric-samples/fraud-detection/

[10] A. W. Services, "Power real-time vector search capabilities with amazon memorydb," AWS Database Blog, 2024, accessed: 2025-04-13. [Online]. Available: https://aws.amazon.com/blogs/database/power-real-time-vector-search-capabilities-with-amazon-memorydb/

[11] M. Wang, J. Dong, Z. Wu, J. Liu, R. Zhang, J. Zhao, R. Wan, X. Lei, S. Zhang, B. Zheng, H. Liu, X. Liao, and H. Jin, "Candor-bench: Benchmarking in-memory continuous anns under dynamic open-world

streams [experiments & analysis]," *Proc. ACM Manag. Data*, vol. 4, no. 1, Apr. 2026. [Online]. Available: https://doi.org/10.1145/3786630

[12] D. Liu, B. Zheng, Z. Yue, F. Ruan, X. Zhou, and C. S. Jensen, "Wolverine: Highly efficient monotonic search path repair for graph-based ann index updates," *Proc. VLDB Endow.*, vol. 18, no. 7, p. 2268–2280, Mar. 2025. [Online]. Available: https://doi.org/10.14778/3734839.3734860

[13] B. Coleman, S. Segarra, A. J. Smola, and A. Shrivastava, "Graph reordering for cache-efficient near neighbor search," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 38 488–38 500.

[14] Z. Yue, B. Zheng, L. Xu, K. Xu, S. Zhang, Y. Du, Y. Gao, X. Zhou, and C. S. Jensen, "Select edges wisely: Monotonic path aware graph layout optimization for disk-based ann search," *Proc. VLDB Endow.*, vol. 18, no. 11, p. 4337–4349, Jul. 2025. [Online]. Available: https://doi.org/10.14778/3749646.3749697

[15] M. Yang, W. Li, J. Jin, X. Zhong, X. Wang, Z. Shen, W. Jia, and W. Wang, "Effective and general distance computation for approximate nearest neighbor search," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 1098–1110.

[16] L. Deng, P. Chen, X. Zeng, T. Wang, Y. Zhao, and K. Zheng, "Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 3, p. 812–821, Nov. 2024. [Online]. Available: https://doi.org/10.14778/3712221.3712244

[17] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, "Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search," 2021. [Online]. Available: https://arxiv.org/abs/2105.09613

[18] Y. Pan, J. Sun, and H. Yu, "Lm-diskann: Low memory footprint in disk-native dynamic graph-based ann indexing," in *2023 IEEE International Conference on Big Data (BigData)*, 2023, pp. 5987–5996.

[19] S. Yang, J. Xie, Y. Liu, J. X. Yu, X. Gao, Q. Wang, Y. Peng, and J. Cui, "Revisiting the index construction of proximity graph-based approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 6, p. 1825–1838, Feb. 2025. [Online]. Available: https://doi.org/10.14778/3725688.3725709

[20] N. Ono and Y. Matsui, "Relative nn-descent: A fast index construction for graph-based approximate nearest neighbor search," in *Proceedings of the 31st ACM International Conference on Multimedia*, ser. MM '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1659–1667. [Online]. Available: https://doi.org/10.1145/3581783.3612290

[21] S. Voruganti and M. T. Özsu, "Mirage-anns: Mixed approach graph-based indexing for approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 3, no. 3, Jun. 2025. [Online]. Available: https://doi.org/10.1145/3725325